

Developmental AI — Technical Reference

Compressed chat notes for Claude Code

Purpose: This document captures the full technical discussion about building a Developmental AI system using a Symbolic + Neural hybrid world model. It covers architecture decisions, tool choices, key papers, hardware requirements, and open implementation questions. Use as context when continuing work in Claude Code.

1. Project Goal

Build an AI agent that learns the way a human child does — driven by curiosity, building progressively from simple to complex skills, with no fixed training dataset and no externally-defined reward function. The target architecture is a **Symbolic + Neural hybrid world model** (approach #3 in the world model taxonomy).

1.1 Why Not Just an LLM?

- LLMs are black boxes — weights emerge from training but cannot be inspected or reasoned about.
- LLMs have no curiosity or intrinsic drive — they don't self-direct learning.
- LLMs are trained once on a static dataset — they don't grow from experience.
- Goal: genuine *developmental* learning, not pattern matching on pre-existing text.

1.2 The Developmental AI Loop (core cycle)

Environment → Curiosity Engine → Set Own Goal → Explore & Act → Observe Result → Prediction Error → Update World Model → Add Skill → (repeat, no endpoint).

Key difference from neural networks: there is no 'correct answer' to aim for. High prediction surprise IS the learning signal (intrinsic reward). Skills compound — learning to grasp enables learning to stack.

2. Chosen Architecture: Symbolic + Neural Hybrid World Model

The world model has THREE layers operating simultaneously:

Layer	What it does	Tool	Description
Sensorimotor	Low-level physics & perception	Gymnasium + DreamerV3	Raw observations, actions, physical consequences
Object/Entity	Things exist, have properties	Neural networks	Symbolic facts: 'cup is on table', action rules, prerequisites
Abstract	Goals, language-level concepts	Language models	Compressed learned representations

Why not a pure simulation / sandbox? Too expensive to simulate every physical detail; sim-to-real gap is severe; can't write rules for abstract concepts. **Why not pure latent space?** Struggles with precise physical reasoning and compositionality (understanding a 'purple elephant' without seeing one).

The winning insight from WALL-E 2.0 (NeurIPS 2025): Use an LLM as the neural backbone but align it with symbolic knowledge (action rules, knowledge graphs, scene graphs) extracted from the agent's own exploration trajectories. This is the closest existing system to the target architecture.

3. Complete Software Stack

All tools are open source. Ray RLLib wraps the entire system for orchestration and scaling.

Component / Tool	Role in Diagram	Install	Notes
Gymnasium (Farama)	Environment — sends observations, receives actions	<code>pip install gymnasium</code>	Standardised RL env API; replaces OpenAI Gym
Curiosity Baselines + ICM	Curiosity Engine — novelty scoring, intrinsic reward	<code>pip install openai/baselines/curiosity</code>	Intrinsic Curiosity Module; prediction error = reward
Stable-Baselines3	Policy — goal-conditioned RL algorithm (PPO/SAC)	<code>pip install stable-baselines3</code>	Beginner-friendly; graduate to RLLib at scale
DreamerV3	World Model core — RSSM encoder/decoder, actor-critic	<code>pip install dreamerv3</code>	Trained on single V100; fixed hyperparams across all domains
Neo4j (Community)	Knowledge Graph — stores symbolic facts and loads	<code>pip install neo4j</code>	Query with Cypher; free community edition
PyKEEN	Graph embeddings — neural/symbolic graph	<code>pip install pykeen</code>	40+ KG embedding models; bridges neural ↔ symbolic
Neo4j GraphRAG	Auto-builds knowledge graph from textual logs	<code>pip install neo4j-graphrag</code>	Populates Neo4j from raw trajectories via NLP
PyTorch Geometric	GNN bridge — structured knowledge fed back into world model	<code>pip install torch-geometric</code>	Learns over graph topology
Skill Bank (HF Hub)	Saves/retrieves learned policies	<code>pip install huggingface_hub</code>	Custom glue code required — no ready library
Ray RLLib	Orchestration, distributed training, scaling	<code>pip install ray[rllib]</code>	Wraps entire stack; scales from laptop to cluster
PyTorch	Foundation — tensor computation for neural components	<code>pip install torch</code>	Required by DreamerV3, PyTorch Geo, SB3, Curiosity Baselines

3.1 Data Flow Between Components

- Gymnasium → ICM + SB3: raw observations
- ICM → DreamerV3: curiosity/intrinsic reward signal
- SB3 → DreamerV3: imagined value estimates for policy
- SB3 → Gymnasium: actions to execute
- DreamerV3 → Neo4j: symbolic facts extracted from experience
- Neo4j → PyKEEN → PyTorch Geometric: graph embeddings
- PyTorch Geometric → DreamerV3: structured knowledge fed back into world model
- DreamerV3 ↔ Skill Bank: bidirectional — saves new skills, loads prior skills for initialisation

3.2 The Custom Glue Layer (what you must build)

No existing library connects DreamerV3’s latent world model to the skill bank and back into the policy layer. This is the novel contribution requiring custom Python code:

- Skill detection logic: when has the agent mastered a capability well enough to 'save' it?
- Skill retrieval: how to select relevant prior skills when setting a new goal?
- Curriculum management: how to escalate challenge complexity as skills compound?
- Neo4j ↔ DreamerV3 bridge: extracting symbolic facts from latent trajectory data

4. Prior Art — What Already Exists

Nobody has combined ALL four layers (curiosity + neural world model + symbolic KG + skill bank) as a unified developmental system. Partial combinations exist:

System	Covers	Missing	Link
WALL-E 2.0 (NeurIPS 2025)	Neural world model + symbolic KG	Curiosity engine, skill bank	github: elated-sawyer/WALL-E

Plan2Explore	DreamerV2/V3 + curiosity (disaggregated, delayed intrinsic reward)	GitHub: ramanans1/plan2explore
DreamerV3-XP (2025)	DreamerV3 + intrinsic reward from symbolic layer (disaggregated)	ResearchGate (Oct 2025)
NeSyS (2026)	Neural world model + symbolic rules (50% data reduction)	GitHub: tianyi-lab/NeSyS
CURIOS (ICML 2019)	Intrinsically motivated modular world model, symbolic layer	arxiv: 1810.06284
HIMA	Hierarchical intrinsically motivated symbolic world model	PMC: 8976870

Target system = WALL-E 2.0 + Plan2Explore + NeSyS, unified. This combination is the novel research contribution.

5. Key Papers to Read

Paper	Year	Why It Matters	Where
Attention Is All You Need	2017	Foundation of all modern LLMs (Transformers)	arxiv: 1706.03762
DreamerV3 (Hafner et al.)	2025	Core world model; trains on single V100; fixed replay buffer	Nature 2025; arxiv: 2301.04104
WALL-E 2.0 (Zhou et al.)	2025	Neurosymbolic world model; closest prior art	NeurIPS 2025; arxiv: 2504.15785
Plan2Explore	2021	DreamerV2 + curiosity; self-supervised world model	arxiv: 2005.05960
Autotelic Agents (Colas et al.)	2020	Agent represents, generates, pursues own goals	arxiv: 2012.09830
CURIOS (Fournier et al.)	2019	Intrinsically motivated modular goal RL	ICML 2019; arxiv: 1810.06284
NeSyS	2026	Neuro-symbolic world model; 50% data reduction	GitHub: tianyi-lab/NeSyS
H-GRAIL (2025)	2025	Hierarchical intrinsic motivations, open-ended	arxiv: 2506.18454
ICM (Pathak et al.)	2017	Original Intrinsic Curiosity Module algorithm	arxiv: 1705.05363
Neurosymbolic Scene Graphs	2023	NSR improves SGG models by up to 33%	Various; see PyKEEN docs

6. Hardware & Environment Requirements

6.1 Coding & Prototyping (no special hardware needed)

- Any modern laptop, 16 GB RAM, standard SSD
- Linux or macOS preferred; Windows via WSL2
- Python 3.11+, CUDA toolkit (if local GPU), Docker
- No GPU required for writing code and testing logic with toy environments

6.2 Training Requirements by Stage

Stage	Hardware	Cost	Use case
Prototyping	Google Colab free (T4, 12h sessions) or Kaggle (P100, 30h/week)	Free	Verify code runs; toy environments
Development	RTX 3080/4070 (10-12 GB VRAM) or Vast.ai	\$0.15-0.50/hr cloud	ICM + DreamerV3 together; simple envs
Serious training	RTX 4090 (24 GB) or Lambda Labs A100	\$1.10/hr (Lambda A100)	Full stack on complex environment
Research scale	Multi-GPU A100/H100 cluster	\$4-8/hr	Production runs, ablations

DreamerV3 reference: Original paper results achieved on a single V100 GPU. Replay buffer = 1,000,000 transitions; batch size = 16; expect 24-72 hrs per serious run. Budget \$25-80 per training run on Lambda Labs A100.

Storage: Plan for 50-200 GB fast SSD for replay buffers and checkpoints.

Smart path: Free Colab/Kaggle → Vast.ai \$0.15/hr for iteration → Lambda Labs A100 at \$1.10/hr for final runs. Lambda offers 50% academic discount.

7. Suggested Implementation Roadmap

Step	Task	Tools	Difficulty	Time
1	Set up Neo4j, store/query a knowledge graph	Neo4j Community	Easy	Weekend
2	Add PyKEEN — train embeddings over PyKEEN	PyKEEN	Medium	1-2 weeks
3	Run DreamerV3 on a basic Gymnasium	DreamerV3 + Gymnasium	Medium	1-2 weeks
4	Add ICM curiosity to the DreamerV3 loop	Curiosity Baselines	Medium	1-2 weeks
5	Build Neo4j ↔ DreamerV3 bridge (symbolic grounding)	Pythonatic (optional) + custom	Hard	1-2 months
6	Build skill bank logic (detect, save, retrieve skills)	ImageHub + custom	Hard	1-2 months
7	Integrate full loop under Ray RLlib	Ray RLlib	Medium	2-4 weeks
8	Scale to complex open-ended environment	Fullstack	Very hard	Ongoing

The 70% that exists: DreamerV3 + Curiosity Baselines + Gymnasium cover most of the loop out of the box. **The 30% you build:** symbolic grounding bridge and skill bank logic — this is where the novel research contribution lives.

8. Key Concepts Quick Reference

Neural Network Basics

- **Weights:** The only thing training changes — billions of adjustable numbers.
- **Backpropagation:** Traces which weights caused an error; nudges them toward correctness.
- **Gradient descent:** Repeated small steps downhill in error space to find good weights.
- **ReLU:** Activation function: if value < 0, set to 0. Introduces non-linearity.
- **Black box problem:** Nobody chose the final weight values — they emerged from training.

Developmental AI Concepts

- **Intrinsic reward:** Curiosity-based reward from prediction surprise — no human labels needed.
- **Prediction error:** Gap between what the world model expected and what actually happened. High surprise = strong learning signal.
- **World model:** Agent's internal compressed model of how the world works — used to IMAGINE future scenarios before acting.
- **RSSM (DreamerV3):** Recurrent State Space Model — the architecture that encodes experience into latent space and imagines futures.
- **Latent space:** Compressed mathematical representation of the world — not a visible 3D space but an abstract geometry of patterns.
- **Open-ended learning:** No fixed endpoint — agent keeps growing indefinitely, driven by curiosity.
- **Skill compounding:** Learning to grasp → enables learning to stack → enables learning to build. Earlier skills scaffold later ones.

Symbolic + Neural Hybrid Concepts

- **Knowledge graph:** Explicit symbolic facts stored as nodes and edges: 'cup → on → table', 'fire → causes → heat'.

- **Graph embedding (PyKEEN):** Converts symbolic graph structure into neural vectors the world model can process.
 - **GNN (PyTorch Geometric):** Neural network that learns patterns OVER graph topology, not just individual nodes.
 - **Neurosymbolic:** Combining neural (fuzzy, learned) with symbolic (hard rules, explicit facts) for best of both worlds.
 - **Scene graph:** Real-time snapshot of current environment as a symbolic graph — bridges raw perception and knowledge graph.
-

9. Open Questions / Things to Build in Claude Code

Use these as starting prompts when continuing in Claude Code:

- How do I connect DreamerV3's latent trajectory data to Neo4j to extract symbolic facts automatically?
- What's the cleanest Python interface between PyTorch Geometric and DreamerV3's RSSM encoder?
- How do I detect when a skill has been 'mastered' and trigger saving it to the Hugging Face Hub?
- What Gymnasium environment is the best starting point for testing this full stack with minimal complexity?
- How do I implement a simple curriculum that escalates difficulty as the skill bank grows?
- Write a minimal working example: DreamerV3 + ICM + Gymnasium in a single Python script.
- What version combinations of PyTorch, DreamerV3, PyTorch Geometric, and Neo4j driver are compatible?
- How does Plan2Explore's disagreement-based intrinsic reward compare to ICM — which should I use?

Useful GitHub Repos

- DreamerV3: github.com/danijar/dreamerv3
 - WALL-E 2.0: github.com/elated-sawyer/WALL-E
 - Plan2Explore: github.com/ramanans1/plan2explore
 - Curiosity Baselines: github.com/Improbable-AI/curiosity_baselines
 - NeSyS: github.com/tianyi-lab/NeSyS
 - SimpleDreamer (PyTorch): github.com/kc-ml2/SimpleDreamer
 - Awesome World Models list: github.com/leofan90/awesome-world-models
-

— End of Reference Document —

Generated from a Claude chat session. May 2026.